

Part A: Developing a Multi-Process Application

1 Introduction

In this report, we present Part A of our CS342 project, which evaluates the performance of a multi-process application designed to find the top K values in a large count of input. The program uses N child processes (processing the file in parallel), shared memory, and parallel processing. Experiments were conducted to analyze scalability by varying N and input sizes. A key requirement of Part A is to measure performance (time) by varying the number of N child processes and input sizes.

2 Experimental Setup

2.1 Implementation Overview

Program Name: findmax.c

The parent process reads the input file and distributes integers into N intermediate files. Each child uses a min-heap with size K to receive top K values. Results are written into a shared memory array, at positions reserved for that child. Parent process waits for all child processes to terminate. Merges the $N \times K$ values from shared memory into a single array. Sorts the merged array and selects the global top K values. Writes the results to OUTFILE in descending order. We measure execution time for each experiment (in seconds) using the C library `clock()` in `findmax.c` for evaluating performance for different N values and different input sizes.

2.2 Hardware

System Configuration

OS: Ubuntu 22.04 LTS (64-bit) running in a virtual machine.

CPU: 8 virtual CPU cores allocated to the VM.

2.3 Test Parameters

To evaluate the multi-process program, we designed experiments varying the input size and the number of child processes N, while fixing K to 10000.

2.3.1 Input Files

Generated datasets containing 1M, 5M, and 10M integers using a random number generator.

2.3.2 Number of Child Processes

We tested the following N values to observe performance scaling and degradation:

N = 1 (sequential, baseline)

N = 3, 4, 6 (modest parallelism)

N = 8 (core limited - the number of available cores in VM)

N = 10, 20 (exceeds the core count, to test overhead)

2.3.3 Fixed K value

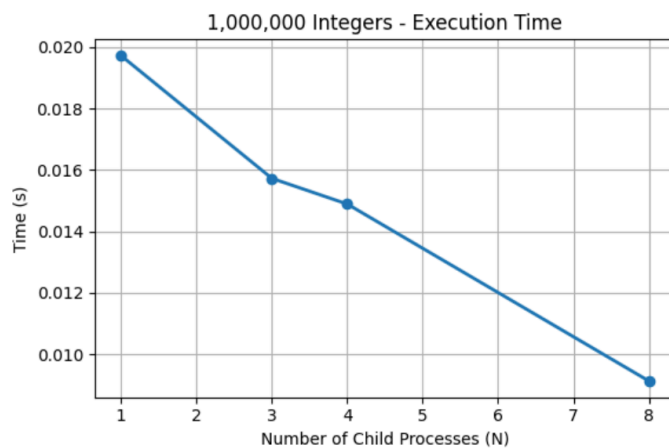
K=10,000 to stress the min-heap logic while ensuring shared memory remains manageable.

2.3.4 Trials and Averages

Each configuration (input size + N) was executed 3 times, and results were averaged.

3 Experiment Results

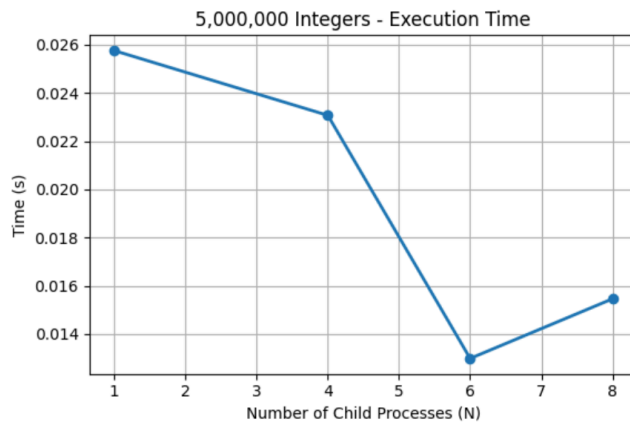
3.1 Results for 1,000,000 Integers and N = 1,3,4,8



N	Time (s)
1	0.019722
3	0.015727
4	0.014891
8	0.009130

Changing N from 1 to 8 better utilizes the 8-core VM that is distributing the workload among multiple processes. Execution time decreases as N increases, peaking at N=8. Near-linear scaling up to N=8, indicating effective utilization of all 8 VM cores.

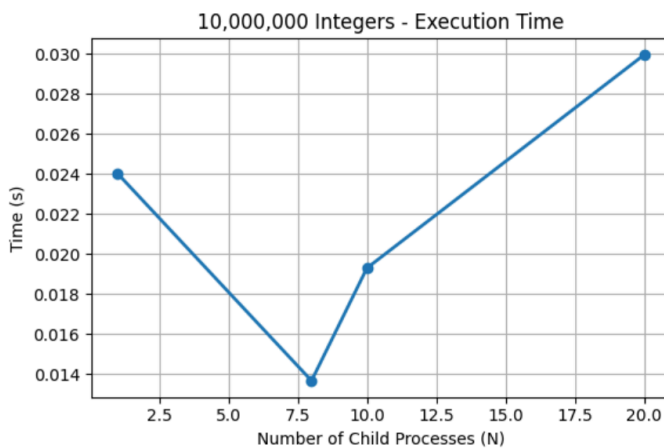
3.2 Results for 5,000,000 Integers and N = 1,3,6,8



N	Time (s)
1	0.025754
4	0.023071
6	0.012987
8	0.015460

In this one, we see a speed up for N go to 6 from 1. However, there is an anomaly that the case for N=8 is a bit slower than N=6. The reason can be: All 8 cores are fully occupied, forcing the OS to prioritize process switching over execution. Since the VM has 8 cores, the OS itself, background processes, and VirtualBox overhead will consume at least 1 or 2 cores. So when N approaches 6 or 7, performance doesn't scale as well. N=6 strikes the best balance between parallelism and overhead. This highlights the importance of reserving CPU resources for system tasks in virtualized environments. In other words, leaving 1-2 cores for system processes can cause better performance for this particular workload.

3.3 Results for 10,000,000 Integers and N = 1, 8, 10, 20



N	Time (s)
1	0.024020
8	0.013654
10	0.019294
20	0.029956

In this experiment we see the optimal performance for N=8 Full Core Utilization: Matches the VM's 8 cores, enabling parallel processing without oversubscription. For N>8 the OS alternates processes on overloaded cores, wasting cycles on context switches. Concurrent memory access from 10–20 processes exceeds the VM's memory subsystem capacity, causing delays. For 10M integers, N=8 maximizes parallelism, while higher N values incur prohibitive overhead. This underscores the limits of scaling processes beyond physical/virtual core counts.

3.4 Observations & Conclusion

3.4.1 N=1 to N=6–8

Parallelization significantly reduces execution time when N approaches the VM's 8-core limit. Each process runs on a dedicated core, minimizing idle CPU time and maximizing throughput.

3.4.2 Performance Degradation at N>8

Exceeding the core count introduces context switching overhead, as the OS time-slices processes across saturated cores. Cache thrashing and memory contention worsen with more processes, slowing memory-bound operations.

3.4.3 Optimal N in Virtualized Environments

N=6 outperforms N=8. Why? The OS, VM, hypervisor, and background tasks consume at least 1–2 cores, leaving 6–7 cores for the application. Leaving cores idle for system processes reduces scheduling conflicts and improves stability.

3.4.4 Memory and Cache Effects

When multiple processes read large arrays simultaneously, cache misses can become more frequent. This is especially noticeable at higher N values (like 10 or 20) where performance can degrade even further.

4 Conclusion

This study evaluates a multi-process application for extracting top K values, demonstrating that parallelization with N child processes improves performance up to the system's core limit. On an 8-core VM, increasing N from 1 to 6–8 reduced runtime significantly. However, exceeding core counts degraded performance due to context-switching overhead and memory contention, with N=20 becoming slower than sequential execution. Notably, N=6 outperformed N=8 for 5M inputs, as system tasks consumed 1–2 cores, underscoring the need to reserve resources in virtualized environments. These results validate that optimal parallelism balances hardware utilization and overhead, recommending N near core counts while monitoring CPU usage. Also we note that in some cases more time needed can result in more input size, comparing all for N = 1 for the input size with 1000000 and 5000000.